path. The `WebHost` property enables you to do that if you really have a good reason to. In the following listing, the Content folder is configured as a replacement for wwwroot.

Listing 2.6 Configuring the static files location

```
builder.WebHost.UseWebRoot("content");
```

The `Services` property provides the entry point to the dependency injection container, which is a centralized place for application services. You will explore dependency injection in more detail in chapter 7, but for the meantime, it is enough to know that the container is responsible for managing the lifetime of application services and providing instances to any part of the application, as needed. The default template includes the following line of code, which makes the essential services the Razor Pages infrastructure relies on available to the application:

```
builder.Services.AddRazorPages();
```

Those services include the Razor view engine, model binding, request validation, tag helpers, memory cache, and `ViewData`. Don’t worry if these terms seem unfamiliar. You will encounter them in much more detail as you work through the book. The important thing to note is that the `Services` property provides you with a place to register and configure additional services as you need them.

Sometimes these services will be parts of the framework that you choose to enable (like the Razor Pages example), and sometimes they represent services that you install as separate packages. Often, they will be services that you write yourself that hold the application logic, like getting and saving data.